

JAVA METHOD

메소드 기본 문법 · 호출 · 오버로딩

메소드는 왜 사용할까?

반복되는 코드를 하나로 묶어 재사용하고, 프로그램을 기능 단위로 나누어 가독성과 유지보수성을 높이기 위해 사용한다.

1. 메소드(Method)란?

특정 작업을 수행하는 코드를 하나의 이름으로 묶어 놓은 기능 단위.

반복 코드 제거 및 재사용

기능별 코드 분리 → 코드가 읽기 쉬워짐

수정이 필요할 때 한 곳만 고치면 되어 유지보수가 쉬움

2. 메소드 정의 기본 문법

```
접근제한자 리턴형 메소드이름(파라미터) {
    // 메소드 호출 시 수행할 코드
    return 리턴값; // 리턴형이 void 이면 생략
}
```

3. 구성 요소

구성 요소	의미	예시
접근제한자	누가 이 메소드를 사용할 수 있는지 결정	public, protected, (default), private
리턴형	메소드 실행 후 호출한 곳으로 돌려주는 값의 자료형	void, int, String, 객체형 등
메소드 이름	메소드를 구별하는 이름. 보통 소문자로 시작	add(), getName()
파라미터	메소드 호출 시 전달받을 값을 선언	int money, String name

접근제한자 한눈에 보기

접근제한자	사용 범위
public	어디서든 접근 가능
protected	같은 패키지 + 상속받은 자식 클래스에서 접근 가능
default	같은 패키지 안에서만 접근 가능 (아무것도 쓰지 않음)
private	현재 클래스 내부에서만 접근 가능

4. 리턴형과 파라미터 이해하기

리턴형

void : 돌려주는 값이 없음 → 코드 실행 후 끝
 기본자료형 : int, double, boolean, char 등
 객체형 : String, 배열, 직접 만든 클래스 객체 등

```
public int add(int a, int b) {
    return a + b;
}

public void printHello() {
    System.out.println("Hello");
}
```

파라미터(parameter)

메소드가 실행될 때 외부에서 전달받을 값을 선언하는 부분이다. 호출할 때 실제로 넣는 값은 argument(인자/인수)라고 한다.

```
public void introduce(String name, int age) {
    System.out.println(name + " / " + age);
}

introduce("승록", 20); // "승록", 20은 전달하는 값(argument)
```

5. 메소드 사용(호출)

정확한 개념

`new Test()`는 클래스를 메모리에 올리는 명령이라기보다, Test 클래스를 설계도로 사용해 객체(instance)를 생성하는 표현이다. 일반 인스턴스 메소드는 그 객체를 통해 호출한다.

```
class Test {
    public void add() {
        System.out.println("add 실행");
    }
}

Test t = new Test(); // Test 객체 생성
t.add();             // 객체를 통해 메소드 호출
```

호출 순서

순서	코드	의미
1	Test t = new Test();	Test 객체를 생성하고 변수 t가 그 객체를 가리킴
2	t.add();	t가 가리키는 객체의 add() 메소드를 실행

예외: static 메소드

static 메소드는 객체에 소속된 메소드가 아니라 클래스에 소속된 메소드이므로, 객체를 만들지 않고 클래스 이름으로 호출할 수 있다.

```
class Calculator {
    public static int add(int a, int b) {
        return a + b;
    }
}

int result = Calculator.add(10, 20); // new 없이 호출
```

6. 메소드 오버로딩(Method Overloading)

하나의 클래스 안에서 같은 이름의 메소드를 여러 개 정의하는 것.

핵심 조건

메소드 이름은 같아도 파라미터의 개수, 자료형 또는 순서가 달라야 한다. 리턴형만 다르게 만드는 것은 오버로딩이 아니다.

가능한 오버로딩

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }

    double add(int a, double b) {
        return a + b;
    }

    double add(double a, int b) {
        return a + b;
    }
}
```

호출할 때는 어떻게 구분할까?

컴파일러가 전달된 인자의 개수와 자료형을 보고 알맞은 메소드를 선택한다.

```
Calculator c = new Calculator();

c.add(1, 2);        // add(int, int)
c.add(1, 2, 3);     // add(int, int, int)
c.add(1.5, 2.5);    // add(double, double)
```

오버로딩이 성립하는 기준

구분 기준	오버로딩 가능?	예시
파라미터 개수	O	add(int, int) / add(int, int, int)
파라미터 자료형	O	add(int, int) / add(double, double)
파라미터 순서	O	add(int, double) / add(double, int)
리턴형만 변경	X	int add(int,int) / double add(int,int) → 불가
파라미터 이름만 변경	X	add(int a) / add(int x) → 불가

7. 시험·필기용 핵심 정리

한 줄 암기

메소드 = 특정 기능을 수행하는 코드 묶음 / 오버로딩 = 이름은 같고 파라미터 구성이 다른 메소드 여러 개

일반 인스턴스 메소드 호출: 객체 생성 → 객체.메소드()
 static 메소드 호출: 클래스이름.메소드()
 void 는 리턴값 없음, void 가 아니면 보통 return 값이 필요함
 private 의 “나”는 현재 클래스라고 이해하면 됨
 오버로딩 판별 시 리턴형은 구분 기준이 아님

주의할 표현

헛갈리기 쉬운 표현	더 정확한 표현
new 로 클래스를 메모리에 올린다	new 로 객체(instance)를 생성한다
파라미터 = 호출할 때 넣는 값	파라미터는 선언부 변수, 실제 넣는 값은 인자(argument)
리턴형이 다르면 오버로딩 가능	파라미터 구성이 달라야 오버로딩 가능